

SBA301 — SLOT 8

BÀI TẬP / DỰ ÁN TỔNG HỢP

PRODUCT REST API DESIGN & INSPECTION KIT

Client-Server • HTTP • JSON • REST • json-server • Postman • DevTools

Mental model: Client → HTTP Request → REST Endpoint → Mock Server → HTTP Response → JSON → Client kiểm tra kết quả.

Thông tin	Giá trị
Course	SBA301 – Integrate Single Page Application with Spring Boot
Lớp	SE1910
Slot	08 – Chapter 07: Client-Server Communication
Bài tập	Product REST API Design and Inspection Kit
Độ khó	Trung bình
Thời lượng	2–4 giờ thực hành
Hình thức	Cá nhân hoặc nhóm nhỏ
Công cụ chính	IntelliJ IDEA / Terminal, Node 21, json-server, Postman, Chrome/Edge DevTools
Trọng tâm	HTTP methods, status codes, headers, JSON, REST endpoint design, API contract
Không thuộc core	Fetch/Axios chuyên sâu, React data fetching, Spring Boot backend hoàn chỉnh

Chu trình thực hành: DESIGN API → CREATE JSON → START MOCK SERVER → SEND REQUEST → INSPECT RESPONSE → BREAK/FIX → DOCUMENT → EXPLAIN

0. THÔNG TIN DỰ ÁN

Hạng mục	Nội dung
Tên dự án	Product REST API Design and Inspection Kit
Mục đích	Củng cố Client–Server, HTTP, JSON, REST; chuẩn bị Slot 9 và Spring Boot REST API
Resource chính	Product
Mock backend	json-server + db.json
Kiểm thử	Postman + Browser DevTools
Thời lượng	2–4 giờ
Sản phẩm nộp	Project folder, db.json, api-contract.md, README.md, evidence screenshots, Postman collection nếu có

Ranh giới kiến thức: Core của Slot 8 là hiểu và kiểm chứng giao tiếp HTTP/JSON/REST. Không biến bài này thành ứng dụng React gọi API bằng Fetch/Axios; phần đó thuộc Slot 9.

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

Sinh viên xây dựng một mock REST API cho tài nguyên Product, tạo dữ liệu JSON, chạy server cục bộ, sau đó dùng Postman và Browser DevTools để gửi/quan sát các request GET, POST, PUT, PATCH, DELETE. Song song, sinh viên lập API contract và giải thích request/response dựa trên method, URL, headers, body, status code và JSON.

1.1 Mục tiêu kiến thức

- Phân biệt client và server trong một request thực tế.
- Đọc được cấu trúc HTTP request/response.
- Mapping CRUD với GET/POST/PUT/PATCH/DELETE.
- Giải thích được các status code 200, 201, 204, 400, 404 và sự khác biệt giữa status code của mock server với API production.
- Tạo và kiểm tra JSON hợp lệ; phân biệt JSON string với JavaScript object.
- Thiết kế REST endpoint theo resource, dùng danh từ và method đúng ngữ nghĩa.
- Hiểu API contract là điểm thống nhất giữa frontend và backend.

1.2 Mục tiêu kỹ năng

- Khởi tạo project Node tối thiểu trong IntelliJ.
- Cài/chạy json-server và xử lý port.
- Tạo db.json và seed data.
- Tạo Postman collection, request body và header.
- Quan sát Network tab trong DevTools.
- Ghi evidence, reset dữ liệu sau thao tác destructive, viết README/API contract.

Kết quả đầu ra: Hoàn thành bài khi mock API chạy, CRUD được kiểm tra, evidence đầy đủ và sinh viên có thể giải thích ít nhất một request từ lúc gửi đến lúc nhận JSON response.

2. CHỨC NĂNG CỦA BÀI TẬP

#	Chức năng	Kết quả chính	Kiến thức Slot 8
1	Thiết kế API	Bảng endpoint Product	REST, resource, endpoint
2	Mock data	db.json hợp lệ	JSON object/array
3	Mock server	localhost:3001	Client–Server
4	GET list	GET /products → 200	GET, response array
5	GET detail	GET /products/{id}	Path/id, 200/404
6	POST create	Tạo product mới	POST, body, 201

#	Chức năng	Kết quả chính	Kiến thức Slot 8
7	PUT update	Thay thế toàn bộ record	PUT, idempotency
8	PATCH update	Thay đổi một vài field	PATCH, partial update
9	DELETE	Xóa record	DELETE, status
10	Postman inspection	Headers/body/status	HTTP request/response
11	DevTools inspection	Network evidence	Browser client
12	JSON parsing	parse/stringify demo	JSON <-> JS object
13	API contract	docs/api-contract.md	Frontend–backend agreement
14	Reset data	Khôi phục seed sau destructive test	Reproducibility
15	AI verification	Trace workflow + critique	AI-assisted learning

3. CÔNG NGHỆ VÀ CÔNG CỤ SỬ DỤNG

Công nghệ / công cụ	Vai trò	Dùng trong bài
Node.js 21	Runtime/tooling	npm, npx, chạy scripts
npm	Quản lý package	npm init, npm install, npm run
json-server	Mock REST API	Tạo endpoint từ db.json
JSON	Định dạng dữ liệu	db.json, body request/response
Postman	API client	CRUD request + evidence
Chrome/Edge DevTools	Quan sát Network	GET bằng browser, headers/response
IntelliJ IDEA	IDE/Terminal	Soạn file, chạy scripts
JavaScript	JSON demo/reset script	JSON.parse/stringify, fs.copyFileSync
Markdown	Documentation	api-contract.md, README.md

Ghi chú phiên bản: Để lab tái lập ổn định với cú pháp `--watch`, tải liệu pin json-server 0.17.4 khi cài local. Nếu lớp dùng phiên bản khác, giảng viên cần kiểm tra CLI trước và điều chỉnh lệnh chạy.

4. CẤU TRÚC THƯ MỤC DỰ ÁN

Cấu trúc đề xuất

```
slot8-product-rest-api-lab/
├── docs/
│   └── api-contract.md
├── evidence/
│   ├── devtools/
│   └── postman/
├── scripts/
│   ├── parse-json-demo.js
│   └── reset-db.js
├── db.json
├── db.seed.json
├── package.json
└── README.md
```

Thành phần	Mục đích
db.json	Database giả lập mà json-server đọc/ghi trong khi test.
db.seed.json	Bản seed sạch để reset sau POST/PUT/PATCH/DELETE.
docs/api-contract.md	Mô tả method, endpoint, body, response và status code kỳ vọng.
scripts/parse-json-demo.js	Thực hành JSON.parse() / JSON.stringify().

Thành phần	Mục đích
scripts/reset-db.js	Copy seed → db.json để tái lập test.
evidence/	Lưu ảnh chụp Postman/DevTools theo checkpoint.
README.md	Cách cài, chạy, reset, test và phạm vi bài lab.

5. YÊU CẦU CÀI ĐẶT VÀ CHUẨN BỊ MÔI TRƯỜNG

5.1 Kiểm tra Node, npm và npx

Terminal trong IntelliJ (Alt+F12)

```
node -v
npm -v
npx --version
```

Yêu cầu môn học dùng Node 21 hoặc phiên bản tương đương. Nếu các lệnh không nhận diện, kiểm tra PATH hoặc cài lại Node trước khi tiếp tục.

5.2 Chuẩn bị IntelliJ và Postman

- Tạo/mở một thư mục trống bằng File > Open.
- Mở Terminal: Alt+F12 hoặc View > Tool Windows > Terminal.
- Cài Postman desktop hoặc dùng công cụ API tương đương theo chính sách lớp.
- Chrome/Edge: chuẩn bị mở F12 > Network để quan sát request.

5.3 Chiến lược cài json-server

Cài local để lab ổn định

```
npm init -y
npm install -D json-server@0.17.4
```

Phương án thay thế: Nếu giảng viên yêu cầu dùng npx trực tiếp theo handout, có thể dùng `npx json-server --watch db.json --port 3001`; tuy nhiên nên thống nhất phiên bản cho cả lớp để kết quả CLI/status nhất quán.

6. TÓM TẮT DANH SÁCH CÁC BƯỚC THỰC HIỆN

Bước	Nội dung	Milestone
1	Tạo/mở project	V0
2	Kiểm tra Node/npm/npx	V0
3	Khởi tạo package.json	V1
4	Cài json-server + scripts	V1
5	Tạo cấu trúc thư mục	V1
6	Tạo db.seed.json và db.json	V2
7	Tạo JSON parse/stringify demo	V2
8	Tạo reset-db.js	V2
9	Chạy mock REST API	V3
10	GET /products bằng browser	V4
11	Quan sát request bằng DevTools	V4
12	Tạo Postman collection	V5
13	GET list và GET detail	V5
14	Test resource not found	V5

Bước	Nội dung	Milestone
15	POST create	V6
16	PUT full update	V6
17	PATCH partial update	V6
18	DELETE + verify	V6
19	Reset seed data	V6
20	Viết API contract	V7
21	Break-it HTTP/JSON lab	V7
22	Tổng hợp test matrix/evidence	V8
23	Viết README	V8
24	Final verification + Git checkpoint	V9

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

Bước 1. Tạo và mở project

Mục	Nội dung
Mục tiêu	Có thư mục làm việc độc lập cho Slot 8.
Thao tác	Trong File Explorer tạo `slot8-product-rest-api-lab`; IntelliJ > File > Open > chọn thư mục. Mở Terminal bằng Alt+F12.
Kết quả mong đợi	IntelliJ hiển thị đúng thư mục root; Terminal đang đứng tại root project.
Cách kiểm tra	Chạy `pwd` (Git Bash/macOS/Linux) hoặc `cd` (Windows CMD) và xác nhận tên thư mục.
Lỗi thường gặp	Mở nhầm parent folder; Terminal đứng ngoài project.

Trạng thái project: V0 – Có thư mục project.

Bước 2. Kiểm tra Node/npm/npx

Mục	Nội dung
Mục tiêu	Xác nhận máy có tooling trước khi cài package.
Thao tác	Chạy ba lệnh kiểm tra phiên bản.
Kết quả mong đợi	Các lệnh trả phiên bản, không báo command not found.
Cách kiểm tra	So sánh output với yêu cầu môi trường lớp.
Lỗi thường gặp	Node chưa vào PATH; Terminal cũ chưa reload PATH.

Lệnh kiểm tra

```
node -v
npm -v
npx --version
```

Mẹo: Nếu vừa cài Node, đóng/mở lại IntelliJ để Terminal nhận PATH mới.

Bước 3. Khởi tạo package.json

Mục	Nội dung
Mục tiêu	Tạo metadata project và nền tảng cho npm scripts.

Mục	Nội dung
Thao tác	Trong Terminal tại root chạy `npm init -y`, sau đó mở package.json trong IntelliJ.
Kết quả mong đợi	Có file package.json hợp lệ.
Cách kiểm tra	Chạy `npm pkg get name version` hoặc mở file kiểm tra.
Lỗi thường gặp	Chạy lệnh ở sai thư mục; package.json nằm nhầm cấp.

Trạng thái project: V1 – Có package.json.

Terminal

```
npm init -y
```

Bước 4. Cài json-server và cấu hình npm scripts

Mục	Nội dung
Mục tiêu	Có mock server ổn định và lệnh chạy nhất quán.
Thao tác	Chạy `npm install -D json-server@0.17.4`. Sau đó sửa phần scripts trong package.json như mẫu.
Kết quả mong đợi	node_modules có json-server; package.json có scripts `api`, `json-demo`, `reset-db`.
Cách kiểm tra	Chạy `npm ls json-server` và kiểm tra không có lỗi đỏ.
Lỗi thường gặp	Network/proxy làm npm install lỗi; package.json sai dấu phẩy khi sửa thủ công.

Trạng thái project: V1 – Tooling đã sẵn sàng.

Cài package

```
npm install -D json-server@0.17.4
```

package.json – bản hoàn chỉnh

```
{
  "name": "slot8-product-rest-api-lab",
  "version": "1.0.0",
  "private": true,
  "description": "SBA301 Slot 8 - Client-Server, HTTP, JSON, REST lab",
  "scripts": {
    "api": "json-server --watch db.json --port 3001",
    "json-demo": "node scripts/parse-json-demo.js",
    "reset-db": "node scripts/reset-db.js"
  },
  "devDependencies": {
    "json-server": "0.17.4"
  }
}
```

Bước 5. Tạo cấu trúc thư mục

Mục	Nội dung
Mục tiêu	Tách data, docs, scripts và evidence rõ ràng.
Thao tác	Trong Project panel tạo `docs`, `evidence/devtools`, `evidence/postman`, `scripts`.
Kết quả mong đợi	Project tree giống cấu trúc ở Phần 4.
Cách kiểm tra	Refresh Project panel và kiểm tra không tạo folder nhầm trong node_modules.
Lỗi thường gặp	Tạo nested folder sai cấp; đặt evidence trong node_modules.

Trạng thái project: V1 – Cấu trúc project hoàn chỉnh.

Bước 6. Tạo db.seed.json và db.json

Mục	Nội dung
Mục tiêu	Có dữ liệu JSON hợp lệ và bản seed để phục hồi.
Thao tác	Tạo `db.seed.json` ở root, dán JSON mẫu; copy cùng nội dung thành `db.json`.
Kết quả mong đợi	Hai file tồn tại, resource root tên `products`, có 4 records.
Cách kiểm tra	IntelliJ không báo JSON syntax error. Có thể chạy `npm run json-demo` sau bước 7 hoặc khởi động server ở bước 9.
Lỗi thường gặp	Trailing comma; key không có nháy kép; id bị trùng; db.json không có object root.

Trạng thái project: V2 – Có dữ liệu mock hợp lệ.

db.seed.json và db.json

```
{
  "products": [
    {
      "id": 1,
      "name": "iPhone 15",
      "category": "Phone",
      "price": 25000000,
      "quantity": 10,
      "active": true
    },
    {
      "id": 2,
      "name": "Samsung Galaxy S24",
      "category": "Phone",
      "price": 22000000,
      "quantity": 15,
      "active": true
    },
    {
      "id": 3,
      "name": "Dell XPS 13",
      "category": "Laptop",
      "price": 35000000,
      "quantity": 5,
      "active": false
    },
    {
      "id": 4,
      "name": "Logitech MX Master 3S",
      "category": "Accessory",
      "price": 2500000,
      "quantity": 20,
      "active": true
    }
  ]
}
```

Mẹo: Không chỉnh trực tiếp db.seed.json trong lúc test CRUD; coi đây là “golden seed”.

Bước 7. Tạo demo JSON.parse / JSON.stringify

Mục	Nội dung
Mục tiêu	Phân biệt JSON text và JavaScript object bằng thực nghiệm.
Thao tác	Tạo `scripts/parse-json-demo.js`, nhập code mẫu và chạy `npm run json-demo`.
Kết quả mong đợi	Console in raw JSON, object đã parse, name/price và chuỗi stringify.
Cách kiểm tra	Thử xóa dấu nháy kép trong JSON string để quan sát SyntaxError, sau đó khôi phục.

Mục	Nội dung
Lỗi thường gặp	Nhầm JSON string với object; sửa JSON sai cú pháp nhưng không đọc vị trí lỗi.

Trạng thái project: V2 – Hiểu JSON ở mức thao tác.

scripts/parse-json-demo.js

```
const productJson = `
{
  "id": 1,
  "name": "iPhone 15",
  "category": "Phone",
  "price": 25000000,
  "quantity": 10,
  "active": true
}
`;

console.log("1) Raw JSON text:");
console.log(productJson);

const productObject = JSON.parse(productJson);
console.log("2) JavaScript object:");
console.log(productObject);
console.log("Name:", productObject.name);
console.log("Price:", productObject.price);

const jsonString = JSON.stringify(productObject, null, 2);
console.log("3) JSON.stringify result:");
console.log(jsonString);
```

Chạy

```
npm run json-demo
```

Bước 8. Tạo script reset database

Mục	Nội dung
Mục tiêu	Có cách tái lập dữ liệu sau các test destructive.
Thao tác	Tạo `scripts/reset-db.js` theo code mẫu. Chưa chạy khi server đang ghi file; sẽ dùng sau bước 18.
Kết quả mong đợi	Script có thể copy seed → db.json.
Cách kiểm tra	Tạm sửa một field trong db.json, chạy `npm run reset-db`, xác nhận dữ liệu trở lại seed.
Lỗi thường gặp	Sai path; chạy reset khi một công cụ khác đang khóa file.

Trạng thái project: V2 – Project có reset strategy.

scripts/reset-db.js

```
const fs = require("fs");
const path = require("path");

const root = path.resolve(__dirname, "..");
const seed = path.join(root, "db.seed.json");
const db = path.join(root, "db.json");

fs.copyFileSync(seed, db);
console.log("db.json has been reset from db.seed.json");
```

Chạy reset

```
npm run reset-db
```

Bước 9. Chạy mock REST API

Mục	Nội dung
Mục tiêu	Khởi động server cục bộ và quan sát endpoint mà json-server tạo.
Thao tác	Terminal 1 chạy `npm run api`. Giữ terminal này mở.
Kết quả mong đợi	Console thông báo server đang chạy ở http://localhost:3001 và resource `/products`.
Cách kiểm tra	Mở browser `http://localhost:3001/products`.
Lỗi thường gặp	Port 3001 bị chiếm; db.json sai cú pháp; package chưa cài.

Trạng thái project: V3 – Mock server đang chạy.

Terminal 1

```
npm run api
```

Mẹo: Nếu port bận, dừng process cũ hoặc đổi port, nhưng phải cập nhật Postman và tài liệu evidence cho nhất quán.

Bước 10. Kiểm tra GET /products bằng trình duyệt

Mục	Nội dung
Mục tiêu	Quan sát client đơn giản nhất gửi GET và nhận JSON.
Thao tác	Chrome/Edge mở `http://localhost:3001/products`.
Kết quả mong đợi	Trình duyệt hiển thị JSON array có 4 products.
Cách kiểm tra	Reload trang; xác nhận URL, dữ liệu và không có 404.
Lỗi thường gặp	Gõ sai port/path; server chưa chạy.

Trạng thái project: V4 – GET list hoạt động.

URL

```
http://localhost:3001/products
```

Bước 11. Quan sát GET bằng Browser DevTools

Mục	Nội dung
Mục tiêu	Đọc request/response thật thay vì chỉ nhìn JSON trên trang.
Thao tác	F12 > Network > Clear > Reload. Chọn request `products`; xem Headers và Response.
Kết quả mong đợi	Xác định được Request URL, Method GET, Status 200, Content-Type và JSON response.
Cách kiểm tra	Chụp ảnh `evidence/devtools/01-get-products.png` với URL/method/status visible.
Lỗi thường gặp	Network bị filter; mở DevTools sau khi request đã chạy nhưng không reload; chọn nhầm request favicon.

Trạng thái project: V4 – Có DevTools evidence.

Mẹo: Sinh viên phải nói được: Browser là client; json-server là server; request là GET; response chứa status + headers + JSON body.

Bước 12. Tạo Postman collection

Mục	Nội dung
Mục tiêu	Tổ chức API tests theo một collection có thể tái sử dụng.
Thao tác	Postman > New Collection > đặt tên `SBA301 Slot8 Product API`. Tạo collection variable `baseUrl` = `http://localhost:3001` nếu phiên bản hỗ trợ.
Kết quả mong đợi	Collection tồn tại và có thể tạo request dùng `{{baseUrl}}/products`.
Cách kiểm tra	Tạo một request GET tạm và kiểm tra URL được resolve thành `http://localhost:3001/products`.
Lỗi thường gặp	Variable chưa chọn đúng scope; baseUrl thừa dấu `/` gây URL có `//`.

Trạng thái project: V5 – Postman workspace sẵn sàng.

Base URL

http://localhost:3001

Bước 13. Kiểm tra GET list và GET detail bằng Postman

Mục	Nội dung
Mục tiêu	So sánh response collection với response một resource.
Thao tác	Tạo request `GET Products` → `{{baseUrl}}/products`; request `GET Product 1` → `{{baseUrl}}/products/1`. Send từng request.
Kết quả mong đợi	GET list trả array; GET detail trả object của id 1; status 200.
Cách kiểm tra	Lưu request vào collection và chụp evidence có method, URL, status, body.
Lỗi thường gặp	Chọn nhầm POST; server chưa chạy; dùng `/api/products` dù mock server đang expose `/products`.

Trạng thái project: V5 – GET list/detail verified.

Bước 14. Kiểm tra trường hợp resource không tồn tại

Mục	Nội dung
Mục tiêu	Phân biệt endpoint route tồn tại nhưng id không có dữ liệu.
Thao tác	Tạo `GET Missing Product` → `{{baseUrl}}/products/999999` và Send.
Kết quả mong đợi	Nhận phản hồi not found (thường 404 với json-server 0.17.4).
Cách kiểm tra	Ghi status, body và giải thích khác với sai port hoặc server down.
Lỗi thường gặp	Cho rằng 404 luôn nghĩa là “URL route không tồn tại”; không phân biệt missing resource.

Trạng thái project: V5 – Có negative test GET.

Bước 15. POST – Tạo Product mới

Mục	Nội dung
Mục tiêu	Thực hành request có body JSON và kiểm tra state server thay đổi.
Thao tác	Postman: POST `{{baseUrl}}/products` > Body > raw > JSON > dán body mẫu. Kiểm tra Content-Type rồi Send.
Kết quả mong đợi	Response trả record mới với id do server gán; thường status 201. GET list sau đó thấy record mới.
Cách kiểm tra	Chụp request body + status + response; chạy GET list để xác nhận persistence.

Mục	Nội dung
Lỗi thường gặp	Body không phải JSON; gửi string cho price; chọn Text thay vì JSON; thiếu Content-Type.

Trạng thái project: V6 – POST create hoạt động.

POST body

```
{
  "name": "MacBook Air M3",
  "category": "Laptop",
  "price": 30000000,
  "quantity": 8,
  "active": true
}
```

Mẹo: Không tự gán id trong POST nếu muốn quan sát cơ chế server tạo id trong mock API.

Bước 16. PUT – Cập nhật toàn bộ Product

Mục	Nội dung
Mục tiêu	Hiểu PUT là biểu diễn đầy đủ mới của resource trong bài lab.
Thao tác	Postman: PUT `{{baseUrl}}/products/1`, Body raw JSON theo mẫu. Send rồi GET lại `/products/1`.
Kết quả mong đợi	Record id 1 có toàn bộ field theo body mới.
Cách kiểm tra	So sánh trước/sau; kiểm tra các field không gửi có thể bị mất trong semantics của full replacement.
Lỗi thường gặp	Gửi body thiếu field nhưng vẫn kỳ vọng giữ toàn bộ field như PATCH.

Trạng thái project: V6 – PUT verified.

PUT body

```
{
  "id": 1,
  "name": "iPhone 15 Pro",
  "category": "Phone",
  "price": 29000000,
  "quantity": 7,
  "active": true
}
```

Bước 17. PATCH – Cập nhật một phần Product

Mục	Nội dung
Mục tiêu	Phân biệt partial update với full update.
Thao tác	Postman: PATCH `{{baseUrl}}/products/1`, chỉ gửi price và quantity. Sau đó GET lại product 1.
Kết quả mong đợi	Chỉ các field gửi lên thay đổi; name/category/active vẫn giữ.
Cách kiểm tra	Đối chiếu response và GET sau PATCH.
Lỗi thường gặp	Dùng PUT nhưng gửi body partial; nhầm PATCH là “POST lần hai”.

Trạng thái project: V6 – PATCH verified.

PATCH body

```
{
  "price": 28000000,
}
```

```
"quantity": 9
}
```

Bước 18. DELETE – Xóa Product và xác minh

Mục	Nội dung
Mục tiêu	Hiểu DELETE thay đổi resource collection và cần verify hậu điều kiện.
Thao tác	Chọn một id an toàn (ví dụ id 4). Gửi DELETE `{{baseUrl}}/products/4`; sau đó GET `/products/4` và GET list.
Kết quả mong đợi	DELETE thành công; GET id 4 sau đó not found; list không còn id 4. Status DELETE có thể khác theo version/tool, vì vậy ghi status thực tế.
Cách kiểm tra	Lưu evidence DELETE, sau đó gửi GET `/products/4` và GET list để kiểm tra hậu điều kiện.
Lỗi thường gặp	Chỉ nhìn status DELETE mà không verify; xóa nhầm record vừa dùng cho evidence khác.

Trạng thái project: V6 – Full CRUD verified.

Mẹo: API testing tốt luôn kiểm tra hậu điều kiện: POST → GET lại, UPDATE → GET lại, DELETE → GET lại.

Bước 19. Reset dữ liệu về seed

Mục	Nội dung
Mục tiêu	Đảm bảo mọi sinh viên/nhóm có trạng thái dữ liệu giống nhau trước khi tổng kết.
Thao tác	Dừng json-server bằng Ctrl+C. Chạy `npm run reset-db`. Chạy lại `npm run api` và GET list.
Kết quả mong đợi	db.json trở về 4 records ban đầu.
Cách kiểm tra	So sánh db.json với db.seed.json và GET `/products`.
Lỗi thường gặp	Reset khi vẫn có thao tác ghi đang diễn ra; quên restart server.

Trạng thái project: V6 – Dữ liệu đã tái lập.

Reset workflow

```
Ctrl+C
npm run reset-db
npm run api
```

Bước 20. Viết API contract

Mục	Nội dung
Mục tiêu	Chuyển các thao tác đã test thành hợp đồng có thể bàn giao cho frontend/backend.
Thao tác	Tạo `docs/api-contract.md` theo mẫu; điền status/body thực tế từ evidence nếu khác kỳ vọng.
Kết quả mong đợi	Tài liệu có base URL, schema, 6 operations và error observations.
Cách kiểm tra	Một bạn khác đọc contract có thể tự tạo request đúng mà không mở source.
Lỗi thường gặp	Endpoint dùng động từ; body/status viết mơ hồ; nhầm React route với API endpoint.

Trạng thái project: V7 – Có API contract.

docs/api-contract.md

```
# Product API Contract
```

Base URL: `http://localhost:3001`

Method	Endpoint	Mô tả	Request body	Response thành công	Status kỳ vọng
GET	/products	Lấy danh sách	Không	Product[]	200
GET	/products/{id}	Lấy theo id	Không	Product	200 / 404
POST	/products	Tạo mới	ProductCreate JSON	Product mới	201
PUT	/products/{id}	Cập nhật toàn bộ	Product JSON	Product cập nhật	200
PATCH	/products/{id}	Cập nhật một phần	Partial Product JSON	Product cập nhật	200
DELETE	/products/{id}	Xóa product	Không	Phụ thuộc mock server	200/204 tùy phiên bản

Product schema used in this lab

- id: number
- name: string
- category: string
- price: number > 0
- quantity: integer >= 0
- active: boolean

Error observations

- Resource id không tồn tại: ghi status/body thực tế.
- JSON body sai cú pháp: ghi status/body thực tế.
- Server không chạy: đây là lỗi kết nối, không phải HTTP 404.

Bước 21. Break-It Lab – cố tình tạo lỗi HTTP/JSON

Case	Thao tác phá lỗi	Quan sát cần ghi	Bài học
B1	GET /product thay vì /products	Status/body hoặc empty result tùy server	Path phải đúng contract
B2	POST body thiếu dấu ngoặc kép	Postman/server báo parse error	JSON syntax khác JS object
B3	POST với Content-Type sai	Quan sát server xử lý ra sao	Header là metadata có ý nghĩa
B4	GET khi server đã Ctrl+C	Browser/Postman connection refused	Lỗi network ≠ HTTP status
B5	GET /products/999999	404/missing resource	Endpoint tồn tại nhưng resource không tồn tại
B6	Dùng GET để “xóa”	Không được thay đổi state theo thiết kế	Method phải đúng semantics

Human-first debugging: Trước khi hỏi AI: ghi Method + URL + Headers + Body + Status/Network error + Response + giả thuyết nguyên nhân. Sau đó mới dùng AI để kiểm tra giả thuyết.

Bước 22. Lập Test Matrix và Evidence Package

ID	Request/Test	Expected	Evidence
T01	GET /products	200 + array	postman/01-get-list.png
T02	GET /products/1	200 + object	postman/02-get-id.png
T03	GET missing id	404/not found	postman/03-get-missing.png
T04	POST /products	201 + new record	postman/04-post.png
T05	PUT /products/1	200 + full update	postman/05-put.png
T06	PATCH /products/1	200 + partial update	postman/06-patch.png
T07	DELETE /products/4	success + verify missing	postman/07-delete.png
T08	Browser GET + Network	GET 200 + headers/response	devtools/01-get-products.png
T09	npm run json-demo	Parse/stringify output	terminal-json-demo.png
T10	npm run reset-db	Seed restored	terminal-reset.png

Evidence rule: Ảnh phải đủ thông tin để thẩm định: request name/method/URL, status và body/headers liên quan; không chỉ chụp màn hình “đẹp”.

Bước 23. Viết README

Mục	Nội dung
Mục tiêu	Một sinh viên khác có thể clone/copy project và tự chạy lab.
Thao tác	Tạo README.md theo mẫu và điều chỉnh nếu lớp dùng command/version khác.
Kết quả mong đợi	README có scope, requirements, install, run, reset, required tests và evidence locations.
Cách kiểm tra	Đóng project rồi đọc README như người mới: có đủ lệnh để chạy từ đầu hay không?
Lỗi thường gặp	README chỉ ghi “npm run api” nhưng không nói install; không ghi reset; không ghi phạm vi Slot 8.

Trạng thái project: V8 – Project có tài liệu vận hành.

README.md

```
# SBA301 Slot 8 - Product REST API Design and Inspection Kit

## Scope
Practice Client-Server, HTTP, JSON, REST, Postman and DevTools.
This lab does NOT implement a real Spring Boot backend and does NOT teach Fetch/Axios in depth.

## Requirements
- Node.js 21 or equivalent course environment
- npm
- Postman
- Chrome/Edge DevTools

## Install
```bash
npm install
```

## Run mock API
```bash
npm run api
```
Base URL: http://localhost:3001

## Run JSON demo
```bash
npm run json-demo
```

## Reset data
Stop json-server first, then:
```bash
npm run reset-db
npm run api
```

## Required API tests
GET /products
GET /products/1
GET /products/999999
POST /products
PUT /products/1
PATCH /products/1
DELETE /products/4

## Evidence
See evidence/postman and evidence/devtools.
See docs/api-contract.md for the API contract.
```

Bước 24. Final verification, build evidence và Git checkpoint

| Mục | Nội dung |
|----------|---|
| Mục tiêu | Khóa phiên bản bài nộp, đảm bảo có thể tái lập và giải thích. |

| Mục | Nội dung |
|------------------|---|
| Thao tác | Reset seed; chạy API; thực hiện lại T01, T04, T08; kiểm tra docs/evidence. Nếu dùng Git: `git status`, add, commit. |
| Kết quả mong đợi | Project chạy từ clean state; evidence khớp contract; không còn file tạm không cần thiết. |
| Cách kiểm tra | Xóa node_modules rồi chạy `npm install`, `npm run api` để xác nhận package.json đủ. Kiểm tra Git log nếu có. |
| Lỗi thường gặp | Nộp node_modules; quên db.seed.json; evidence từ trạng thái dữ liệu khác contract. |

Trạng thái project: V9 – Bài tập tổng hợp hoàn chỉnh.

Git checkpoint

```
git status
git add .
git commit -m "Complete Slot 8 HTTP JSON REST lab"
git log -1
```

Mẹo: Không bắt buộc nộp node_modules. Nộp package.json/package-lock.json để tái cài dependency.

8. ĐÁNH GIÁ KẾT QUẢ HOÀN THÀNH TỪNG BƯỚC

| Bước | Kết quả mong đợi | Cách kiểm tra | Trạng thái |
|------|--------------------------------|-------------------------------|--|
| 1 | Mở đúng project root | Project panel + terminal path | ☐ Đạt ☐ Chưa đạt |
| 2 | Tooling trả version | node/npm/npx -v | ☐ Đạt ☐ Chưa đạt |
| 3 | Có package.json | Mở package.json | ☐ Đạt ☐ Chưa đạt |
| 4 | json-server + scripts sẵn sàng | npm ls + npm scripts | ☐ Đạt ☐ Chưa đạt |
| 5 | Đúng folder tree | Project tree | ☐ Đạt ☐ Chưa đạt |
| 6 | Có seed + db JSON hợp lệ | JSON inspection | ☐ Đạt ☐ Chưa đạt |
| 7 | Parse/stringify chạy | npm run json-demo | ☐ Đạt ☐ Chưa đạt |
| 8 | Reset script chạy | npm run reset-db | ☐ Đạt ☐ Chưa đạt |
| 9 | Server port 3001 | npm run api | ☐ Đạt ☐ Chưa đạt |
| 10 | Browser thấy array JSON | Browser /products | ☐ Đạt ☐ Chưa đạt |
| 11 | Network evidence GET 200 | DevTools Network | ☐ Đạt ☐ Chưa đạt |
| 12 | Có Postman collection | Postman collection | ☐ Đạt ☐ Chưa đạt |
| 13 | GET list/detail 200 | Send GET requests | ☐ Đạt ☐ Chưa đạt |
| 14 | Negative GET được ghi nhận | GET missing id | ☐ Đạt ☐ Chưa đạt |
| 15 | POST tạo record | POST then GET | ☐ Đạt ☐ Chưa đạt |
| 16 | PUT cập nhật toàn bộ | PUT then GET | ☐ Đạt ☐ Chưa đạt |
| 17 | PATCH cập nhật một phần | PATCH then GET | ☐ Đạt ☐ Chưa đạt |
| 18 | DELETE + verify | DELETE then GET | ☐ Đạt ☐ Chưa đạt |
| 19 | Seed được restore | Compare seed/list | ☐ Đạt ☐ Chưa đạt |
| 20 | API contract hoàn chỉnh | Peer reads contract | ☐ Đạt ☐ Chưa đạt |
| 21 | Có log break-it | Break-it worksheet | ☐ Đạt ☐ Chưa đạt |
| 22 | Test matrix/evidence đủ | Check evidence paths | ☐ Đạt ☐ Chưa đạt |
| 23 | README tái lập được | Fresh-run from README | ☐ Đạt ☐ Chưa đạt |
| 24 | Final verification + commit | Clean reinstall + Git | ☐ Đạt ☐ Chưa đạt |

Stop rule: Không chuyển sang Slot 9 nếu sinh viên chưa phân biệt được lỗi network với HTTP 4xx/5xx, chưa đọc được method/status/body hoặc chưa giải thích được JSON response.

9. CHẠY VÀ KIỂM TRA BÀI THỰC HÀNH HOÀN CHỈNH

9.1 Quy trình chạy từ clean state

Terminal

```
npm install
npm run reset-db
npm run api
```

Giữ Terminal chạy API. Mở Postman/Browser ở cửa sổ khác. Nếu cần reset giữa test sets, dừng server trước, reset rồi restart.

9.2 Bộ test case bắt buộc

| STT | Chức năng | Thao tác | Kết quả mong đợi | Đạt/Không đạt |
|-----|--------------|---------------|-------------------------|---------------|
| 1 | Server start | npm run api | Server listen port 3001 | |
| 2 | GET list | GET /products | 200 + JSON array | |

| STT | Chức năng | Thao tác | Kết quả mong đợi | Đạt/Không đạt |
|-----|---------------------|--------------------------------|---|---------------|
| 3 | GET detail | GET /products/1 | 200 + product 1 | |
| 4 | Missing id | GET /products/999999 | Not found / 404 | |
| 5 | POST valid | POST + JSON body | Created / 201; record mới | |
| 6 | POST malformed JSON | Body sai cú pháp | Request bị lỗi; ghi status/body thực tế | |
| 7 | PUT | PUT /products/1 full body | Record 1 full update | |
| 8 | PATCH | PATCH /products/1 partial body | Chỉ field gửi thay đổi | |
| 9 | DELETE | DELETE /products/4 | Xóa thành công | |
| 10 | Verify POST | GET list | Thấy record mới | |
| 11 | Verify DELETE | GET /products/4 | Không còn record | |
| 12 | Header request | POST/PUT/PATCH | Content-Type JSON | |
| 13 | Header response | GET list | Content-Type JSON | |
| 14 | DevTools | Browser /products | Network có URL/method/status/body | |
| 15 | Network failure | Stop server rồi GET | Connection error, không phải 404 | |
| 16 | Reset | npm run reset-db + api | 4 seed records trở lại | |

9.3 Câu hỏi kiểm tra miệng sau test

- Vì sao GET không nên làm thay đổi dữ liệu?
- PUT và PATCH khác nhau như thế nào trong lab này?
- Status code nằm trong request hay response?
- Content-Type nằm ở đâu và có vai trò gì?
- JSON response là text hay JavaScript object?
- Tại sao `/products` trong json-server khác với React route `/products` của Slot 7?
- Nếu thay json-server bằng Spring Boot, API contract nào cần được giữ ổn định cho frontend?

10. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

| Dấu hiệu | Nguyên nhân thường gặp | Cách xử lý | Phòng tránh |
|-------------------------------|----------------------------|---------------------------------------|---------------------------------|
| `node` / `npm` not recognized | Node/PATH chưa đúng | Cài Node; restart IDE/terminal | Check versions trước lab |
| npm install fail | Network/proxy/registry | Kiểm tra mạng/proxy, đọc log đầu tiên | Không xóa log; pin version |
| Port 3001 in use | Process cũ đang chạy | Dừng process hoặc đổi port đồng bộ | Dùng một instance server |
| json-server start fail | db.json invalid/path sai | Validate JSON; chạy tại root | Giữ db.json ở root |
| GET 404 | Path/id không tồn tại | Check URL và data | Dùng contract + negative test |
| POST không như kỳ vọng | Method/body/header sai | Body raw JSON + Content-Type | Tạo request template |
| PUT làm mất field | Body full thiếu field | Gửi đầy đủ representation | Hiểu PUT vs PATCH |
| PATCH không đổi UI/data | Sai id/path/body | GET trước/sau để verify | Tách từng biến số |
| DELETE rồi test khác fail | Data đã bị destructive | Reset seed | Có reset script |
| JSON.parse SyntaxError | JSON text sai cú pháp | Đọc line/column, kiểm tra quote/comma | IDE JSON validation |
| DevTools không thấy request | Filter/cache/timing | Clear filter, reload | Mở Network trước request |
| Connection refused | Server không chạy/port sai | Start server/check port | Phân biệt network vs HTTP error |

Debug order: 1) Server running? → 2) Method + URL đúng? → 3) Headers đúng? → 4) Body JSON hợp lệ? → 5) Status? → 6) Response body? → 7) Dữ liệu trong db.json?

11. PHẦN MỞ RỘNG VỚI AI ASSISTANT

Nguyên tắc AI-0: Trước khi hỏi AI, sinh viên phải tự ghi bằng chứng request/response và một giả thuyết. AI được dùng để giải thích/kiểm tra/refactor, không thay thế thao tác Postman và DevTools.

11.1 Prompt giải thích – 10 mẫu

1. Giải thích mô hình client-server trong lab json-server này và chỉ rõ client, server, resource.
2. Phân tích đầy đủ request/response khi Postman gửi GET /products.
3. Giải thích vì sao POST /products cần body JSON và Content-Type.
4. Giải thích PUT và PATCH bằng dữ liệu Product của project.
5. Giải thích ý nghĩa 200, 201, 404 và vì sao status nằm ở response.
6. Giải thích JSON.parse và JSON.stringify trong parse-json-demo.js.
7. Giải thích vì sao endpoint REST ưu tiên danh từ.
8. Giải thích cách json-server biến key products trong db.json thành endpoint.
9. Giải thích workflow POST /products từ lúc nhấn Send đến lúc db.json thay đổi.
10. Phân biệt lỗi HTTP 404 với lỗi connection refused khi server không chạy.

11.2 Prompt tối ưu API design – 10 mẫu

1. Đánh giá API contract Product của tôi theo REST resource design, không viết lại code.
2. Đề xuất pagination cho GET /products và giải thích query params phù hợp.
3. Đề xuất sorting theo price/name bằng query params.
4. Đề xuất filtering theo category/active, tránh tạo endpoint động từ.
5. Đề xuất format lỗi JSON thống nhất cho backend Spring Boot tương lai.
6. Khi nào dùng path param, query param và request body trong Product API?
7. Đề xuất API versioning nếu v1 thay đổi breaking contract.
8. Đề xuất OpenAPI/Swagger structure tương ứng với contract hiện tại.
9. Chỉ ra những điểm trong mock json-server không đại diện đầy đủ cho production backend.
10. Lập mapping từ contract hiện tại sang Spring Boot
@GetMapping/@PostMapping/@PutMapping/@PatchMapping/@DeleteMapping.

11.3 Prompt trace workflow – 10 mẫu

1. Mô tả workflow browser GET /products theo các bước client → TCP/HTTP → server → response.
2. Mô tả workflow POST /products và dữ liệu nào nằm ở URL/header/body.
3. Mô tả workflow khi GET id không tồn tại và cách client phân biệt lỗi.
4. Mô tả hậu điều kiện cần kiểm tra sau PUT, PATCH, DELETE.
5. Mô tả tác dụng của reset-db.js đối với reproducibility của test.
6. Mô tả cách status code, headers và body phối hợp để client hiểu response.
7. Mô tả điều gì xảy ra nếu Content-Type không đúng nhưng body trông giống JSON.
8. Mô tả sự khác nhau giữa React route /products/1 và REST endpoint /products/1.
9. Mô tả nếu server đổi port từ 3001 sang 8080 thì những nơi nào trong lab cần cập nhật.
10. Mô tả những gì giữ nguyên nếu json-server được thay bằng Spring Boot API.

11.4 Prompt debug – 5 mẫu

1. POST /products không tạo record. Đây là method, URL, headers, body, status và response của tôi: [dán evidence]. Hãy chẩn đoán theo thứ tự nguyên nhân có thể.
2. GET /products/1 trả unexpected result. Hãy phân tích route, id type và db.json từ evidence tôi cung cấp.
3. json-server không start. Hãy đọc log này và chỉ ra lỗi đầu tiên cần sửa, không đề xuất cài lại mọi thứ.
4. JSON.parse báo SyntaxError tại dòng X. Hãy chỉ ra quy tắc JSON bị vi phạm và đưa bản sửa tối thiểu.
5. DELETE trả success nhưng test sau thất bại. Hãy giúp tôi kiểm tra hậu điều kiện và trạng thái dữ liệu.

11.5 AI Learning Log

| Prompt | AI trả lời tóm tắt | Evidence tôi kiểm tra | Quyết định/điều chỉnh | Bài học |
|--------|--------------------|-----------------------|-----------------------|---------|
| | | | | |
| | | | | |
| | | | | |

12. CHECKLIST Củng cố kiến thức Slot 8

| Nội dung tự kiểm tra | Có | Chưa | Cần làm lại |
|---|--------------------------|--------------------------|--------------------------|
| Tôi phân biệt được client và server. | ☐ | ☐ | ☐ |
| Tôi phân biệt request và response. | ☐ | ☐ | ☐ |
| Tôi đọc được method, URL, headers, body. | ☐ | ☐ | ☐ |
| Tôi hiểu GET/POST/PUT/PATCH/DELETE. | ☐ | ☐ | ☐ |
| Tôi hiểu 200/201/204/400/404 và biết status thực tế có thể phụ thuộc mock server. | ☐ | ☐ | ☐ |
| Tôi biết JSON là text format, không đồng nhất với JavaScript object. | ☐ | ☐ | ☐ |
| Tôi dùng được JSON.parse và JSON.stringify. | ☐ | ☐ | ☐ |
| Tôi hiểu resource, endpoint và RESTful API. | ☐ | ☐ | ☐ |
| Tôi thiết kế endpoint bằng danh từ và method đúng ngữ nghĩa. | ☐ | ☐ | ☐ |
| Tôi chạy được json-server. | ☐ | ☐ | ☐ |
| Tôi dùng Postman kiểm tra CRUD. | ☐ | ☐ | ☐ |
| Tôi dùng DevTools xem request/response. | ☐ | ☐ | ☐ |
| Tôi có API contract. | ☐ | ☐ | ☐ |
| Tôi biết reset seed data sau destructive test. | ☐ | ☐ | ☐ |
| Tôi phân biệt network error với HTTP error. | ☐ | ☐ | ☐ |
| Tôi có thể giải thích một request end-to-end không nhìn tài liệu. | ☐ | ☐ | ☐ |
| Tôi biết phạm vi Slot 8 và chưa lần sang Fetch/Axios của Slot 9. | ☐ | ☐ | ☐ |
| Tôi dùng AI sau khi có evidence và tự verify lại. | ☐ | ☐ | ☐ |

12.1 Human Verification Gate – 20 câu

1. Ai là client trong GET từ Browser tới json-server?
2. Status code thuộc request hay response?
3. GET /products và GET /products/1 khác nhau về resource representation thế nào?
4. Vì sao GET không dùng để xóa?
5. POST thường tạo resource mới và PUT thường cập nhật resource thế nào?
6. PATCH khác PUT ở mức body/semantics?
7. Content-Type: application/json nói gì?
8. JSON array và JSON object khác nhau thế nào?
9. JSON.parse nhận gì và trả gì?
10. JSON.stringify nhận gì và trả gì?
11. REST là protocol hay architectural style?
12. Resource trong bài là gì?
13. Endpoint base của bài là gì?
14. 404 missing product khác server down thế nào?
15. Vì sao cần GET lại sau POST/PUT/PATCH/DELETE?

16. Tại sao db.seed.json không nên bị thay đổi khi test?
17. API contract giúp team frontend/backend như thế nào?
18. Nếu chuyển sang Spring Boot, contract nào nên giữ?
19. Slot 9 sẽ thêm kiến thức gì lên trên nền tảng Slot 8?
20. Hãy trace POST /products từ Postman đến response và db.json.

13. RUBRIC ĐÁNH GIÁ BÀI TẬP TỔNG HỢP

| Tiêu chí | Trọng số | Minh chứng |
|-----------------------------------|----------|---|
| Project chạy được với json-server | 15% | npm install + npm run api; browser/Postman GET thành công |
| db.json + seed hợp lệ | 10% | JSON đúng, dữ liệu hợp lý, reset được |
| REST API design đúng resource | 15% | Contract rõ, endpoint danh từ, method đúng |
| Postman GET/POST/PUT/PATCH/DELETE | 20% | Evidence + hậu điều kiện |
| HTTP status/headers analysis | 10% | Giải thích đúng method/status/header/body |
| JSON parse/stringify | 5% | Script chạy + giải thích |
| DevTools inspection | 5% | Network screenshot + trace |
| API contract | 10% | Method/endpoint/body/response/status |
| README/reproducibility | 5% | Người khác chạy theo được |
| Human verification / giải thích | 5% | Trả lời miệng/trace workflow |

13.1 Mức đánh giá

| Mức | Mô tả |
|----------|--|
| Chưa đạt | Mock API chưa chạy hoặc không giải thích được request/response. |
| Đạt | Chạy được, test GET/POST, hiểu cơ bản HTTP/JSON/REST. |
| Khá | Đủ CRUD, API contract, status/header evidence và reset strategy. |
| Tốt | Test có negative cases, giải thích PUT/PATCH/network-vs-HTTP rõ, README tái lập tốt. |
| Xuất sắc | Có evidence chặt chẽ, AI learning log được verify, mở rộng pagination/filtering/API error contract nhưng không lấn phạm vi Slot 9. |

PHỤ LỤC A. QUICK REFERENCE

| Nhu cầu | Method / concept | Ví dụ |
|------------------------|------------------|----------------------------|
| Lấy danh sách | GET | GET /products |
| Lấy chi tiết | GET | GET /products/1 |
| Tạo mới | POST | POST /products + JSON body |
| Cập nhật toàn bộ | PUT | PUT /products/1 |
| Cập nhật một phần | PATCH | PATCH /products/1 |
| Xóa | DELETE | DELETE /products/1 |
| Mô tả body gửi | Content-Type | application/json |
| Thành công lấy dữ liệu | 200 | OK |
| Tạo mới thành công | 201 | Created |
| Không nội dung | 204 | No Content |
| Request không hợp lệ | 400 | Bad Request |

| Nhu cầu | Method / concept | Ví dụ |
|----------------|------------------|-----------------------|
| Không tìm thấy | 404 | Not Found |
| Lỗi server | 500 | Internal Server Error |

REST endpoint design – mẫu đúng/sai

| Chưa tốt | Tốt hơn | Lý do |
|---------------------|-----------------------|---------------------------|
| /getAllProducts | GET /products | Resource + method |
| /getProduct?id=1 | GET /products/1 | ID là resource identifier |
| /createProduct | POST /products | POST biểu đạt create |
| /updateProduct?id=1 | PUT/PATCH /products/1 | Method biểu đạt update |
| /deleteProduct?id=1 | DELETE /products/1 | DELETE biểu đạt delete |

PHỤ LỤC B. WORKSHEET – REQUEST / RESPONSE TRACE

| Trưởng | Sinh viên ghi |
|-------------------|---------------|
| Scenario | |
| Client | |
| Server | |
| Method | |
| URL | |
| Request headers | |
| Request body | |
| Expected status | |
| Actual status | |
| Response headers | |
| Response body | |
| Data changed? | |
| Evidence filename | |
| Conclusion | |

PHỤ LỤC C. BRIDGE TO SLOT 9

Slot 8 trả lời: request cần method nào, URL nào, headers/body nào, server trả status và JSON gì. Slot 9 mới tập trung vào cách JavaScript/React thực sự gửi request bằng Promise, async/await, Fetch/Axios, quản lý loading/error/data và caching. Điều kiện chuyển tiếp: sinh viên phải đọc được request/response và giải thích được REST contract; nếu chưa, quay lại Human Verification Gate.

| Slot 8 đã biết | Slot 9 sẽ thêm |
|----------------------------------|--|
| GET /products + 200 + JSON array | fetch/axios gọi endpoint và đưa data vào state |
| POST + JSON body + Content-Type | async submit form, error handling |
| 404 vs network error | try/catch và xử lý HTTP response |
| API contract | service layer/API client |

| Slot 8 đã biết | Slot 9 sẽ thêm |
|----------------|---|
| JSON response | response.json(), mapping UI |
| Headers/status | interceptor/config/error policy ở mức phù hợp |